

CS218 - Data Structures (DS)

Week – 01

18-24 Aug/2025

Instructor: **Fizza Aqeel**

Objectives

- To understand the design of fundamental data structures and algorithms for problem solving through computer system
- To study the tradeoff choices in the design and implementation of data structures
- To provide a rigorous “hands-on” experience with implementing different data structures in a high-level programming language
- To analyze time/space tradeoff for different solutions to the same problem
- Understand dictionary/search data structures (lists, trees, hash tables).
- Understand graph representations and algorithms.

Objectives (cont..)

- Understand time and space analysis for both iterative and recursive algorithms and be able to prove the correctness a non-trivial algorithm.
- Be able to translate high-level, abstract data structure descriptions into concrete code.
- Understand how real-world problems map to abstract graph problems.
- Be able to communicate clearly and precisely about the correctness and analysis of basic algorithms (both oral and written communication).

Objectives (cont..)

- Data structures will include lists, vectors, queues, stacks, trees, hash-tables, and graphs.
- Algorithms include searching, sorting using the above data structures, iterative, and recursive algorithms
- On assignments, students are asked to implement data structures and algorithms using a high-level programming language.
- On exams, students are asked to solve problems using analytical methods and programming skills acquired from lectures and assignments to demonstrate knowledge and comprehension of course material.

Course Outline

1. Topics to be covered:			
List of Topics	No. of Weeks	Contact Hours	CLO
ADT, C++/ Java Language Specification, Pointers revisited/ pass-by-reference and pass by value, Rule of Three, Dynamic Safe Arrays	1	3	1
List (Singly Linked List), List (Doubly Linked List), List (Circular Linked List), Linear, Binary & Interpolation Search using Arrays and Linked Lists	1	3	1,3
Elementary Sorting Techniques (Bubble sort, Selection Sort, Insertion Sort, Radix Sort, Shell sort, Comb sort)	2	6	1, 3
Recursion, it's types, issues and Backtracking (with examples), Stack, Queue, their implementation strategies and applications (Simulation of recursion)	1	3	1,2,3
===== Mid-term 1 Exam =====			

Course Outline (cont..)

Advanced Sorting Techniques (Merge sort, Quick sort)	1	3	3
Binary trees and their properties (Full Binary Tree, Complete Binary Tree), Binary Search Trees, their operations and applications, skewness and issues	2	7	1, 2, 3
Balance in Binary Search Trees, AVL Trees, 2-3 trees, B-trees	2	3	2, 3
===== Mid-term 2 Exam =====			
Priority Queues, Heaps as Priority Queues, Heap Sort	1	3	1, 3
Hashing, Hash Functions, Collision-resolution Techniques, Rehashing	1	3	1, 3
String search (Brute force, Rabin Karp, Boyer Moore, Knuth Morris)	1	3	4
Minimum Spanning Trees, Graph Algorithms, Topological Sort, Graphs and their representation and traversal, Shortest Path Problem	1	3	4
===== Final Exam =====			
Total	16	48	

Book & References

□ Prerequisites

- OOP

□ Text Book

- Data Structures and Algorithms in C++ 4th Edition- by Adam Drozdek
- Data Structures & Algorithm Analysis in C++ (3rd Edition). Mark Allen Weiss

□ Other Reference

- C++ How to Program, 5th edition, H. M. Deitel, P. J. Deitel

Marks Distribution

Assignments: 10 %

Mid exams: 30 %

Quiz : 10%

Final Exam: 50 %

What this course is about ?

- Data structures: conceptual and concrete ways to organize data for efficient storage and efficient manipulation
- Employment of this data structures in the design of efficient algorithms
- Analysis of the complexity of your algorithms

Why do we need them?

- Requirements for a good software:
 - Clean Design
 - Easy maintenance
 - Reliable (no core dumps)
 - Easy to use
 - Fast algorithms
- **Efficient data structures**
- **Efficient algorithms**

Tentative Topics

Algorithm Analysis

Arrays, Stacks and Queues

Vectors, lists and sequences

Trees

Heaps / Priority Queues

Binary Search Trees –

Search Trees

Hashing / Dictionaries

Sorting

Graphs and graph algorithms

What will you learn?

- What are some of the common data structures
- What are some ways to implement them
- How to analyze the complexity of your algorithms using the data structures
- How to use them to solve practical problems

C++ (OOP) warm-up

- Basic features
- Parameter passing
- Classes
- Inheritance and virtual
- Four Pillars of OOP
- Abstract Data Types
- Header file
- IO
- Memory Management
- Big three: destructor, copy constructor, and assignment operator
- Const
- Template

Object Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm that organizes code around objects, which represent real-world entities. It focuses on using **classes** as blueprints to create **objects** that encapsulate data (attributes) and behaviors (methods).

This approach promotes modularity, reusability, and scalability in software development.

Classes and Objects

- **Class**: a type definition that includes both
 - data properties, and
 - operations permitted on that data
- **Object**: a variable that
 - is declared to be of some Class
 - therefore includes both data and operations for that data
- **Appropriate usage:**
 - “A variable is an instance of a type.”
 - “An object is an instance of a class.”

Class syntax - Example

```
// A class for simulating an integer memory cell

class IntCell
{
    public:
        IntCell( )
        { storedValue = 0; }

        IntCell(int initialValue )
        { storedValue = initialValue; }

        int read( )
        { return storedValue; }

        void write( int x )
        { storedValue = x; }

    private:
        int storedValue;
};
```

} constructors

Class & Object syntax - Example

```
class MyClass {           // The class
    public:               // Access specifier
        int myNum;       // Attribute (int variable)
        string myString; // Attribute (string variable)
};

int main() {
    MyClass myObj; // Create an object of MyClass

    // Access attributes and set values
    myObj.myNum = 15;
    myObj.myString = "Some text";

    // Print attribute values
    cout << myObj.myNum << "\n";
    cout << myObj.myString;
    return 0;
}
```

Data Abstraction

- Providing only essential information to the outside world and hiding their background details, i.e., to represent the needed information in program without presenting the details.
- Hiding details and showing only essential features, implemented using **abstract classes** (with pure virtual functions).

```
#include <iostream>
using namespace std;

class BankAccount {
public: virtual void deposit(double)=0;
       virtual void show()=0;
};

class Savings :
public BankAccount {
    double balance=0;
public:
    void deposit(double amt) override {
        balance+=amt;
    }
    void show() override {
        cout<<"Balance: "<<balance;
    }
};

int main(){
    BankAccount* acc=new Savings();
    acc->deposit(500);
    acc->show(); delete acc;
}
```

Data Encapsulation

Restrict access to data so that it can be manipulated only in authorized ways.

Separate class declarations from implementation (e.g. public, private in C++).

```
#include <iostream>
using namespace std;

class BankAccount {
private:
    double balance; // encapsulated data

public:
    BankAccount() : balance(0) {}

    void deposit(double amount) {
        balance += amount;
    }

    double getBalance() {
        return balance;
    }
};
```

Inheritance

- Reusing features of an existing class in a new class.
- SavingsAccount inherits from BankAccount and gets access to its public methods like deposit() and getBalance().

```
class SavingsAccount : public BankAccount {  
private:  
    double interestRate;  
  
public:  
    SavingsAccount(double rate) : interestRate(rate)  
    {  
    }  
  
    void addInterest() {  
        double interest = getBalance() * interestRate;  
        deposit(interest);  
    }  
};
```

Polymorphism

- One interface, many forms. Achieved by **function overriding** (runtime polymorphism) or **function overloading** (compile-time polymorphism).
- Same function `accountType()` behaves differently depending on the object → polymorphism.

```
BankAccountBase* acc = new Savings();  
acc->accountType(); // prints "Savings Account"
```

```
class BankAccountBase {  
public:  
    virtual void accountType() { // virtual enables  
runtime polymorphism  
        cout << "Generic Bank Account" << endl;  
    }  
};  
  
class Savings : public BankAccountBase {  
public:  
    void accountType() override {  
        cout << "Savings Account" << endl;  
    }  
};  
  
class Current : public BankAccountBase {  
public:  
    void accountType() override {  
        cout << "Current Account" << endl;  
    }  
};
```

UML Class Diagrams

- The Class Diagram for what we describe may be shown as the following box:

Complex
-re:Real -im:Real
<u>+create(in r:Real = 0, in i:Real = 0):Complex</u> +real():Real +imag():Real +abs():Real +exp():Complex +normalize() +destroy()

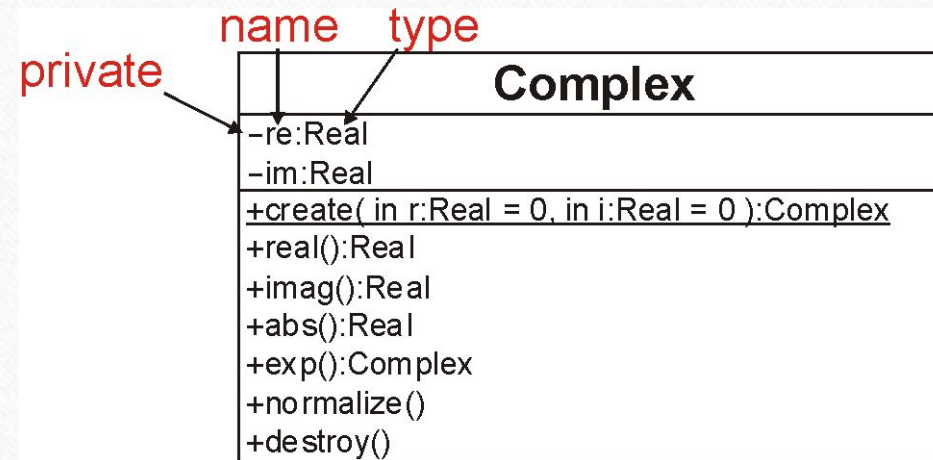
UML Class Diagrams

- The three components include:
 - the name, the attributes, and the operations

Class Name	
Complex	
-re:Real -im:Real	Attributes (class members)
+create(in r:Real = 0, in i:Real = 0):Complex	
+real():Real +imag():Real +abs():Real +exp():Complex +normalize() +destroy()	Operations (member functions, methods, behaviours)

UML Class Diagrams

- The attributes are described by:
 - a visibility modifier, a name, and a type



Visibility

- Visibility in C# and Java is described by placing public/private/protected in front of each class member or member function
- In C++, this is described by a block prefixed by one of
 - **private:**
 - **protected:**
 - **public:**

Accessors and Mutators

We can classify member functions into two categories:

- Those leaving the object unchanged
- Those modifying the member variables of the object

Respectively, these are referred to as:

- **Accessor (Getter)** → A method used to access (read) the value of a private data member.
- **Mutator (Setter)** → A method used to modify (write/change) the value of a private data member.

Accessors and Mutators

```
#include <iostream>
using namespace std;

class Student {
    int age; // private by default
public:
    void setAge(int a) { age = a; } // Mutator
    int getAge() { return age; }    // Accessor
};

int main() {
    Student s;
    s.setAge(21); // using setter
    cout << "Age: " << s.getAge(); // using getter
}
```

//output

Name: Ali
Age: 20

Templates

- A template in C++ is a feature that allows writing generic code. It lets you
- create functions or classes that can work with any data type (int, float, char, string, etc.) without rewriting the same logic for each type.
- In simple words: Templates = Code reusability for different data types.

Templates

That mechanism is called a template:

```
template <typename Type>  
Type sqr( Type x ) {  
    return x*x;  
}
```

- template → keyword
- typename (or class) → tells compiler it's a placeholder for a data type
- Type → the generic type (can be used like a normal datatype)

This creates a function which returns something of the same type as the argument

Templates

Function Template Example

```
#include <iostream>
using namespace std;

// Template function to find maximum of two values
template <typename T>
T getMax(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    cout << getMax(10, 20) << endl;    // int
    cout << getMax(5.5, 2.3) << endl;  // double
    cout << getMax('A', 'Z') << endl;  // char
}
```

```
//output
20
5.5
Z
```

Class Template Example

```
#include <iostream>
using namespace std;

// Template class for a simple Pair
template <class T>
class Pair {
    T first, second;
public:
    Pair(T a, T b) : first(a), second(b) {}
    T getMax() { return (first > second) ? first : second; }
};
```

```
int main() {
    Pair<int> p1(10, 20);
    Pair<double> p2(3.14, 2.71);
    cout << p1.getMax() << endl;  // int
    cout << p2.getMax() << endl;  // double
}
```

```
//output
20
5.5
```

Pointers and const

There are two different ways that pointers and `const` can be intermixed:

1. Constant pointer
2. Pointer to a constant variable

Constant Pointer

- A `const` pointer must be initialized to a value upon declaration, and its value can not be changed.
- However, because the value being pointed to is still non-`const`, it is possible to change the value being pointed to via dereferencing the pointer:

```
int *const p = &i;  
*p = 6; // it is O.K.  
I = 10;  
I++;  
p = &j;    // NOT O.K.
```

Pointer to a const variable

- It is also possible to declare a pointer to a constant variable by using the `const` before the data type:

```
int i;  
const int * p = &i;  
*p = 6; // it is NOT O.K. because i is  
        //treated as constant when accessed by p.  
  
P = &m;  
M = 7;  
*p = 7; //
```

- However, it can be changed independently:

```
i = 6; // It is O.K.
```

- It is also possible to declare a `const` pointer to a constant value:

```
const int n = 5;  
const int * const p = &n;  
*p = 10;  
P = &x;
```

Parameter Passing

In C, all parameters are passed by value (call by value). But C++ offers three options:

- **Call by value**
 - Copy of data passed to function
 - Changes to copy do not change original
- **Call by reference**
 - Uses &
 - Avoids a copy and allows changes to the original
- **Call by constant reference**
 - Uses `const&`
 - Avoids a copy and guarantees that actual parameter will not be changed

Example

```
#include <iostream>
using std::cout;
using std::endl;
```

```
int squareByValue( int ); // pass by value
void squareByReference( int & ); // pass by reference
int squareByConstReference ( const int & ); // const ref.
```

```
int main()
{ int x = 2, z = 4, r1, r2;

  r1 = squareByValue(x);
  squareByReference( z );
  r2 = squareByConstReference(x);

  cout << "x = " << x << " z = " << z << endl;
  cout << "r1 = " << r1 << " r2 = " << r2 << endl;
  return 0;
}
```

Pointers to Functions

static Class Members

- Shared by all objects of a class
 - Normally, each object gets its own copy of each variable
- Efficient when a single copy of data is enough
 - Only the static variable has to be updated
- May seem like global variables, but have class scope
 - Only accessible to objects of same class
- Exist even if no instances (objects) of the class exist
- Can be variables or functions
 - public, private, or protected

Standard Template Library

- I/O Facilities: `iostream`
- Garbage-collected String class
- Containers
 - `vector`, `list`, `queue`, `stack`, `map`, `set`
- Numerical
 - `complex`
- General algorithms
 - `search`, `sort`

Using the vector

- Vector: Dynamically growing, shrinking array of elements
- To use it include library header file:

```
#include <vector>
```

- Vectors are declared as

```
vector<int> a(4); //a vector called a,  
                //containing four integers  
vector<int> b(4, 3); //a vector of four  
                // elements, each initialized to 3.  
vector<int> c; // 0 int objects
```

- The elements of an integer vector behave just like ordinary integer variables
a[2] = 45;

Manipulating vectors

- **The `size()` member function** returns the number of elements in the vector.
`a.size()` returns a value of 4.
- **The `=` operator** can be used to assign one vector to another.
- e.g. `v1 = v2`, so long as they are vectors of the same type.
- **The `push_back()` member function** allows you to add elements to the end of a vector.

push_back() and pop_back()

```
vector<int> v;  
v.push_back(3);  
v.push_back(2);  
// v[0] is 3, v[1] is 2, v.size() is 2  
v.pop_back();  
int t = v[v.size()-1]; //t=3  
v.pop_back();
```

Data Type

- Set of data with predefined values

OR

- Something that can store particular “type” of values

Data Type

- At the top level, there are two types of data types:
 - Primitive Data Types (System-defined Data Types)
 - User-defined Data Types

Primitive Data Types

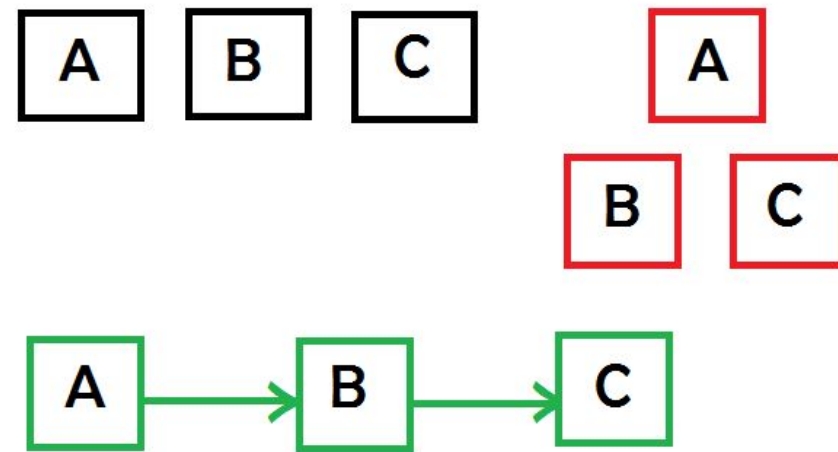
- Data types defined by the system
- Examples are: *int, float, char, double, etc.*

User-defined Data Types

- Most programming languages allow users to define their own data types
- Examples are: *structures in C, classes in C++ & JAVA*

Data Structures

- *Data structure* is a particular way of storing and arranging data (in a computer) so that it can be used efficiently



Abstract Data Type (ADT)

- All primitive data types (int, float etc.) support basic operations like addition, subtraction
- But what about user-defined data types???

ADT

- ADT = Abstract Data Types
- A logical view of the data objects together with specifications of the operations required to create and manipulate them.
- Describe an algorithm – pseudo-code
- Describe a data structure – ADT

Arrays

- Array: a set of pairs (**index** and **value**)

- **data structure**

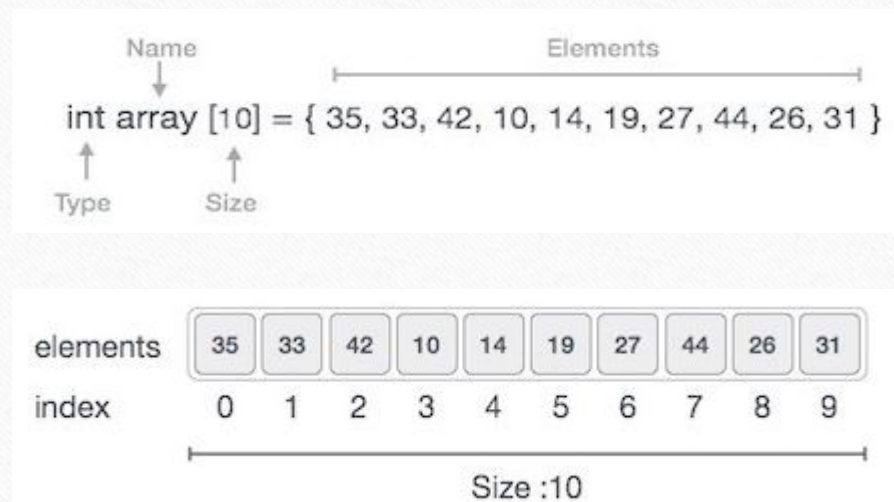
For each index, there is a value associated with that index.

- **representation (possible)**

implemented by using consecutive memory.

Array Representation

- Arrays can be declared in various ways in different languages. For illustration, let's take C++ array declaration.



Example

```
#include <iostream>
void print1(int *ptr, int rows)
{
    int i;
    cout << "Address Contents" << endl;
    for (i=0; i < rows; i++)
        cout << ptr+i << " " << *(ptr+i) << endl;
}
void main(){
    int one[] = {0, 1, 2, 3, 4}; //Goal: print out address and value
    print1(one, 5);
}
```

Address Contents

0xbffffbb4 0

0xbffffbb8 1

0xbffffbbc 2

0xbffffbc0 3

0xbffffbc4 4

Basic Operations

Following are the basic operations supported by an array.

- **Traverse** – print all the array elements one by one.
- **Insertion** – Adds an element at the given index.
- **Deletion** – Deletes an element at the given index.
- **Search** – Searches an element using the given index or by the value.
- **Update** – Updates an element at the given index.

Arrays & Lists

- Two built-in data structures that can be used to organize data, or to create other data structures:
- Lists
- Arrays

List



Array



Lists

- A list is an ordered set of data. It is often used to store objects that are to be processed sequentially.
- A list can be used to create a queue.

List



Array



Arrays

- An array is an indexed set of variables, such as `dancer[1]`, `dancer[2]`, `dancer[3]`,... It is like a set of boxes that hold things.
- A list is a set of items.
- An array is a set of variables that each store an item.

List

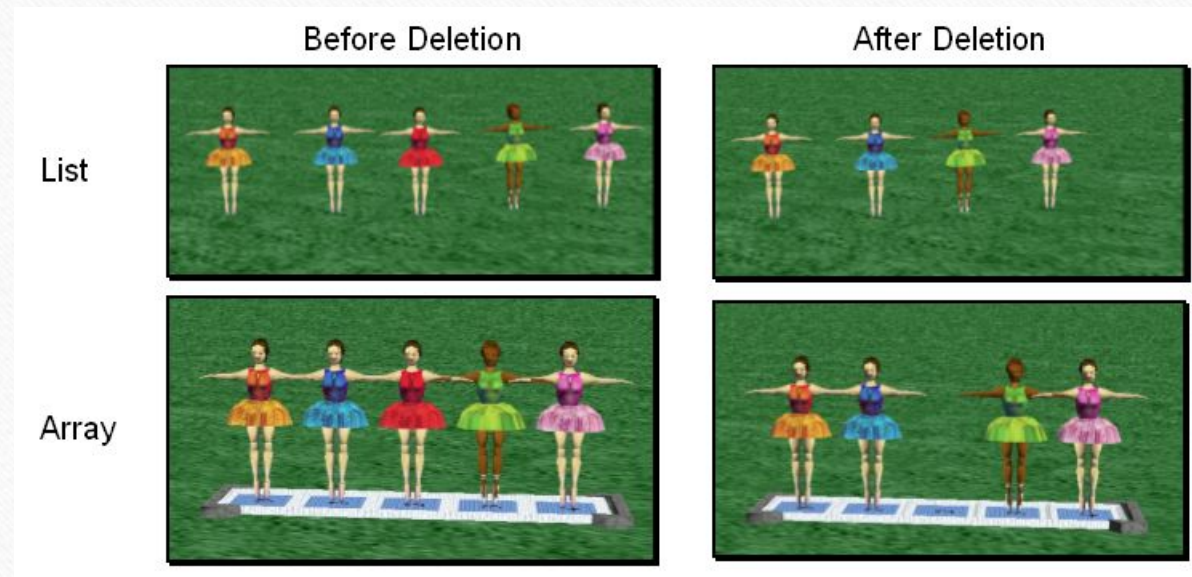


Array



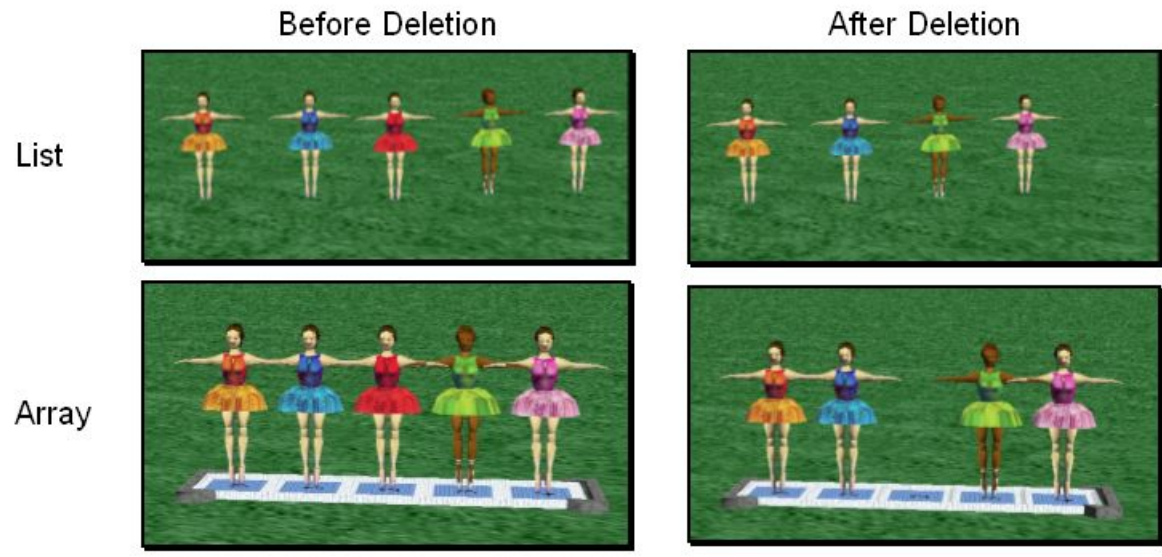
Arrays & Lists

- You can see the difference between arrays and lists when you delete items.



Arrays & Lists

In a list, the missing spot is filled in when something is deleted.



Array of Objects

- An **array of objects** means creating multiple instances of a class and storing them in a single array.
- Each element of the array is an object of the class.
- Useful when we want to handle multiple objects of the same type together (like list of students, employees, etc.).

Array of Objects

```
#include <iostream>
using namespace std;

class Student {
    int roll;
    string name;
public:
    void setData(int r, string n) {
        roll = r; name = n;
    }
    void display() {
        cout << roll << " - " << name << endl;
    }
};
```

```
//continue
int main() {
    Student s[3]; // Array of 3 objects

    s[0].setData(1, "Ali");
    s[1].setData(2, "Sara");
    s[2].setData(3, "Ahmed");

    for(int i=0; i<3; i++)
        s[i].display();
}
```

Rule OF Three in C++

Reading Material Shared!